

OBSERVER 2.0 — Documento descrittivo esteso (Clean LaTeX build)

2026-01-25

Indice

1	000_README_DOCSET.md	1
2	OBSERVER Document Set	1
2.1	Scopo di questo pacchetto	1
2.2	Indice dei documenti	1
2.2.1	Canonici	1
2.2.2	PDF	1
2.2.3	Supporto (ground truth e prove)	1
2.2.4	Specifiche tecniche (spec)	2
2.3	Regole di derivazione (PDR/PDD/DATADictionary)	2
2.4	Convenzioni	2
3	001_PROJECT_OVERVIEW.md	2
4	OBSERVER 2.0 - Documento descrittivo allineato al codice	2
4.1	1. In una frase	2
4.2	2. Il problema che risolve	2
4.3	3. Valore oggi e potenziale domani (taglio retail)	3
4.3.1	3.1 Valore oggi (implementato nello snapshot)	3
4.3.2	3.2 Potenziale domani (roadmap coerente con un retail “serio”)	3
4.3.3	3.3 Valutazione architetturale (perche’ la base e’ solida)	3
4.3.4	3.4 Aree di attenzione (priorita’ di completamento, non implementate nello snapshot)	3
4.3.5	3.5 Scenari di utilizzo ad alta efficacia (research; richiedono validazione)	3
4.3.6	3.6 Piano di completamento suggerito (per derivare backlog implementativo)	3
4.4	4. Applicazione come organismo (componenti e responsabilita’)	4
4.4.1	4.7 Forecast ranking (Wave 6): contratto I/O e algoritmo	4
4.4.2	4.8 Lifecycle Monitor: stati concettuali e mapping UI<->audit	5
4.5	5. Fasi operative (workflow)	5
4.5.1	5.1 Setup e migrazione schema	5
4.5.2	5.2 Run di audit (backtest)	6
4.5.3	5.3 Certify e verify	6
4.5.4	5.4 NEWS-ALPHA lane (segnali da news)	6
4.5.5	5.5 Forecast ranking (Wave 6)	6
4.6	6. Principi (contratti di progettazione)	6
4.6.1	6.1 Offline-by-default (operational safety)	6
4.6.2	6.2 Auditability first	6
4.6.3	6.3 No future leak (regola fondamentale)	6
4.6.4	6.4 Retail realism: costi e fiscalita’	6
4.7	7. Ranking a stelle (Wave 6): logica e parametri	6
4.7.1	7.1 Obiettivo	6
4.7.2	7.2 Formula (sintesi)	7
4.8	8. UI (Streamlit): pagine e utilizzo	7
4.9	9. Derivazione documenti (PDR, PDD, DATADictionary)	7
4.10	10. Gap Register (cio’ che manca o e’ deliberatamente rinviato)	7
4.11	Glossario minimo	7
5	002_PDR_OBSERVER.md	7

6	PDR - OBSERVER (Product / Project Definition Requirements) - v1.0	7
6.1	1. Executive summary	8
6.2	2. Target user e casi d'uso	8
6.2.1	2.1 Persona primaria: Retail avanzato “disciplinato”	8
6.2.2	2.2 Persona secondaria: Builder/quant retail	8
6.2.3	2.3 Casi d'uso principali	8
6.3	3. Scope (in-scope vs out-of-scope)	8
6.3.1	3.1 In-scope (implementato o previsto a breve)	8
6.3.2	3.2 Out-of-scope (per questa release)	8
6.4	4. Requisiti funzionali (FR)	8
6.4.1	FR-01 Offline-by-default	8
6.4.2	FR-02 Audit run tracciabile	8
6.4.3	FR-03 No future leak	8
6.4.4	FR-04 Survivorship bias control	9
6.4.5	FR-05 NEWS-ALPHA scoring deterministico	9
6.4.6	FR-06 Forecast ranking deterministico	9
6.4.7	FR-07 UI di controllo	9
6.4.8	FR-08 Risk controls (future release; non implementato nello snapshot)	9
6.4.9	FR-09 Execution gateway + execution log (future release; non implementato nello snapshot)	9
6.4.10	FR-10 Model monitoring (future release; non implementato nello snapshot)	9
6.5	5. Requisiti non funzionali (NFR)	9
6.6	6. Criteri di accettazione (Definition of Done)	9
6.7	7. Roadmap suggerita (retail + rigore monetario/finanziario)	10
6.8	8. Rischi e mitigazioni	10
7	003_PDD_OBSERVER.md	10
8	PDD - OBSERVER (Product / Project Design Document) - v1.0	10
8.1	1. Design intent	10
8.2	2. Architettura (layer)	10
8.2.1	2.1 Layer operativi (entrypoint)	10
8.2.2	2.2 Layer applicativo (src/)	10
8.2.3	2.3 Layer presentazione (UI)	10
8.3	3. Dataflow (end-to-end)	11
8.4	4. Data layer (DuckDB)	11
8.5	5. Audit engine: regole chiave	11
8.5.1	5.1 No future leak (T+1)	11
8.5.2	5.2 Execution halts	11
8.5.3	5.3 Retail realism	11
8.6	6. Forecast & stars ranking (Wave 6)	11
8.7	7. Operabilit� e packaging	11
8.8	8. Estensioni future (coerenti con trading monetario/finanziario)	11
8.8.1	8.1 RiskEngine (pre-trade + post-trade) — proposta	11
8.8.2	8.2 ExecutionGateway / OMS (paper-first) — proposta	12
8.8.3	8.3 Transaction Cost Analysis + Model Monitoring — proposta	12
8.9	9. Considerazioni di rischio e compliance (retail)	12
9	004_DDT_DATADictionary.md	12
10	DATADictionary (DuckDB) - v1.0	12
10.1	Panoramica	12
10.1.1	Convenzioni generali	12
10.1.2	Relazioni logiche (alto livello)	13
10.2	metadata	13
10.2.1	Colonne	13
10.2.2	Vincoli (DuckDB)	13
10.3	prices	13
10.3.1	Colonne	13
10.3.2	Vincoli (DuckDB)	14
10.4	dividends	14
10.4.1	Colonne	14
10.4.2	Vincoli (DuckDB)	14

10.5	recs	14
10.5.1	Colonne	14
10.5.2	Vincoli (DuckDB)	14
10.6	momentum_rankings	15
10.6.1	Colonne	15
10.6.2	Vincoli (DuckDB)	15
10.7	universes	15
10.7.1	Colonne	15
10.7.2	Vincoli (DuckDB)	15
10.8	universe_membership	15
10.8.1	Colonne	15
10.8.2	Vincoli (DuckDB)	15
10.9	ticker_mappings	16
10.9.1	Colonne	16
10.9.2	Vincoli (DuckDB)	16
10.10	ticker_halts	16
10.10.1	Colonne	16
10.10.2	Vincoli (DuckDB)	16
10.11	market_halts	16
10.11.1	Colonne	16
10.11.2	Vincoli (DuckDB)	17
10.12	sentiment_cache	17
10.12.1	Colonne	17
10.12.2	Vincoli (DuckDB)	17
10.13	data_gaps	17
10.13.1	Colonne	18
10.13.2	Vincoli (DuckDB)	18
10.14	audit_runs	18
10.14.1	Colonne	19
10.14.2	Vincoli (DuckDB)	19
10.15	audit_signal_decisions	19
10.15.1	Colonne	19
10.15.2	Vincoli (DuckDB)	20
10.16	audit_trades	20
10.16.1	Colonne	21
10.16.2	Vincoli (DuckDB)	22
10.17	audit_equity	22
10.17.1	Colonne	22
10.17.2	Vincoli (DuckDB)	22
11	005_TRACEABILITY_MATRIX.md	22
12	Traceability Matrix (Repo -> Documenti) - v1.1	22
12.1	Scopo	22
12.2	Matrice	24
12.3	Regola di accettazione (operativa)	25
13	006_REPO_BOM.md	25
14	Repository BOM (Bill of Materials) - v1.1	25
14.1	Sintesi	25
14.2	Struttura ad alto livello	25
14.3	Runner e utility operative (scripts/)	25
14.4	UI Streamlit (pages/)	25
14.5	Moduli applicativi principali (src/*.py)	26
14.6	Package interni	26
14.7	Note operative (retail)	26
15	007_PARAMETER_SNAPSHOT.md	26
16	Parameter Snapshot - v1.1	26
16.1	Scopo	26

16.2	Note interpretative (retail)	27
17	008_EVIDENCE_PACK.md	27
18	Evidence Pack - v1.1	27
18.1	Obiettivo	27
18.2	1) Setup ambiente	27
18.2.1	Windows (PowerShell)	27
18.2.2	Linux/macOS (bash)	28
18.3	2) Schema e stato DB (DuckDB)	28
18.4	3) Test suite	28
18.5	4) Run / Certify / Verify (SENTINEL-ALPHA)	28
18.5.1	4.0 Postura offline/online (precedenza + contract operativo)	28
18.5.2	4.1 Run (operativo)	28
18.5.3	4.2 Certify (postura audit - offline di default)	28
18.5.4	4.3 Verify (post-run)	28
18.6	5) Forecast ranking (Wave 6)	28
18.7	6) NEWS-ALPHA (lane news)	29
18.7.1	6.1 Status	29
18.7.2	6.2 Collect (RSS) - online guardato	29
18.7.3	6.3 Run (consume fixtures -> DuckDB)	29
18.7.4	6.4 History lane (GDELT)	29
18.8	7) Avvio UI Streamlit	29
18.9	8) Packaging sessione (archiviazione deterministica)	30
18.109	Naming convention (artefatti)	30
18.1110	Checklist di riproducibilit� (retail)	30
19	009_GAP_REGISTER.md	30
20	Gap Register - v1.1	30
20.1	Scopo	30
20.2	1) Implementato (con evidenza nel codice)	30
20.3	2) Parzialmente implementato / disclosure-first	31
20.4	3) Mancante (backlog suggerito)	31
20.4.1	Execution layer (paper/live) + OMS + log ordini	31
20.4.2	Risk management (pre-trade e post-trade)	31
20.4.3	Transaction Cost Analysis (TCA) + model monitoring (drift)	31
20.4.4	Upgrade News/Sentiment (opzionale, se richiesto)	31
20.4.5	Macro/monetary layer (trading avanzato, taglio retail)	31
20.4.6	FX translation e multi-currency	31
20.4.7	Corporate actions end-to-end	31
20.4.8	Execution realism avanzato	31
20.5	4) Documentazione: gap residui	32

Questo documento e' generato automaticamente dai canonici Markdown in docs/.

\newpage

1 000_README_DOCSET.md

2 OBSERVER Document Set

Build date: 2026-01-25

Repository snapshot: included in this package (see SHA256SUMS.txt)

2.1 Scopo di questo pacchetto

Questo ZIP contiene il repository OBSERVER (codice + dati locali + report) e un set di documenti *allineati al codice* e strutturati per consentire la derivazione controllata di:

- **PDR** (Product / Project Definition Requirements): requisiti, scope, criteri di accettazione, roadmap.
- **PDD** (Product / Project Design Document): architettura, scelte progettuali, componenti, trade-off.
- **DATADITIONARY**: modello dati DuckDB (tabelle/colonne/chiavi) con semantica e regole.

L'obiettivo operativo e' trasformare la “descrizione del progetto” in un deliverable **verificabile** (tracciabilita', coerenza, auditability).

2.2 Indice dei documenti

2.2.1 Canonici

2.2.2 PDF

- docs/OBSERVER_documento_descrittivo_esteso_allineato_v1.2.pdf (esteso, con indice reale, tabelle UI/Use-case/Gaps)
- docs/OBSERVER_documento_descrittivo_allineato_v1.1.pdf (snello, overview auditabile)
- 001_PROJECT_OVERVIEW.md
Descrizione end-to-end dell'applicazione (linguaggio retail, ma con rigore da ingegneria quant).
- 002_PDR_OBSERVER.md
Requisiti e criteri di accettazione.
- 003_PDD_OBSERVER.md
Architettura, moduli, scelte di design, pattern di audit.
- 004_DDT_DATADictionary.md
Data model DuckDB: schema + semantica per ogni tabella.

2.2.3 Supporto (ground truth e prove)

- 005_TRACEABILITY_MATRIX.md
Matrice Repo -> Documento (copertura e mappatura funzionale).
- 006_REPO_BOM.md
Inventory del repository (Bill of Materials).
- 007_PARAMETER_SNAPSHOT.md
Parametri/Costanti estratti dal codice (evita divergenze doc-vs-code).
- 008_EVIDENCE_PACK.md
Comandi, artefatti attesi, naming conventions, check di riproducibilita'.
- 009_GAP_REGISTER.md
Gap e backlog: cio' che manca (o e' deliberatamente “planned”).

2.2.4 Specifiche tecniche (spec)

- specs/WAVE6_FORECAST_STARS_RANKING_SPEC.md
- specs/SENTINEL_RUNNER_SPEC.md
- specs/NEWS_ALPHA_SPEC.md
- specs/AUDIT_LIFECYCLE_SPEC.md

2.3 Regole di derivazione (PDR/PDD/DATADictionary)

- **PDR**: deve usare come fonti primarie 001_PROJECT_OVERVIEW + 009_GAP_REGISTER + 008_EVIDENCE_PACK.
Ogni requisito deve avere almeno un riferimento a un componente (vedi matrice 005).
- **PDD**: deve usare come fonti primarie 001_PROJECT_OVERVIEW + specifiche in docs/specs/ + schema in 004_DDT_DATADictionary.
Le scelte progettuali devono essere motivate con vincoli retail (costi, tasse, limiti dati) e vincoli audit (provenance, no future leak).
- **DATADictionary**: deve essere generabile (in futuro) automaticamente da information_schema + arricchita manualmente con semantica.
La sezione “Semantica” non deve contraddire i moduli src/db/migrate.py e src/db/audit_store.py.

2.4 Convenzioni

- Date: YYYY-MM-DD (DATE) e timestamp in UTC quando presenti (TIMESTAMP TZ).
- “Offline-by-default”: nessuna chiamata rete senza flag esplicito --online e/o env.
- “Auditability first”: output deterministici, run_id, report firmati (fingerprint codice).

\newpage

3 001_PROJECT_OVERVIEW.md

4 OBSERVER 2.0 - Documento descrittivo allineato al codice

Build date: 2026-01-25

4.1 1. In una frase

OBSERVER e' un sistema “offline-by-default” che *raccoglie segnali (incluso news-sentiment)*, li sottopone a *gate di qualita'/provenance*, *esegue backtest certificabili senza future-leak* e produce **ranking a stelle** e report leggibili per un investitore retail.

4.2 2. Il problema che risolve

Nel retail avanzato (portafogli azionari/ETF e gestione di notizie), i problemi ricorrenti sono:

- **Backtest non auditabile**: risultati non riproducibili, leakage temporale, universi non controllati (survivorship bias).
- **Dati “sporchi”**: ticker inconsistenti, prezzi mancanti, sospensioni di mercato, corporate actions.
- **Segnali non comparabili**: fonti diverse (firm/rating) senza calibrazione e senza contesto (confidence).
- **Operativita' poco trasparente**: mancano transcript, run_id, fingerprint del codice; difficile capire cosa e' stato eseguito “davvero”.

OBSERVER struttura tutto il ciclo in modo tracciabile e verificabile.

4.3 3. Valore oggi e potenziale domani (taglio retail)

4.3.1 3.1 Valore oggi (implementato nello snapshot)

- **Audit engine “certification-grade”**: regole conservative (es. T+1, no future leak) e ledger per trade/equity.
- **Offline-by-default**: evita dipendenze esterne; online e' un'eccezione esplicita.
- **NEWS-ALPHA**: raccolta, scoring sentiment deterministico, dedup, aggregazione e output riproducibile.
- **Forecast & ranking a stelle** (Wave 6): calibrazione su storico audit_trades + shrinkage + confidence + percentile-to-stars.
- **UI Streamlit**: monitoraggio e controllo pipeline, audit runs, gap e ranking.

4.3.2 3.2 Potenziale domani (roadmap coerente con un retail “serio”)

- Regimi macro/monetari (tassi, inflazione, policy) come “contesto” per modulare rischio e sizing.
- Multi-asset (ETF, FX, rates) e gestione multi-valuta (FX translation).
- Modellazione dividendi e corporate actions end-to-end (cashflow) per total return.
- Execution realism: slippage dinamico, spread per liquidita', limiti di size retail.

4.3.3 3.3 Valutazione architetturale (perche' la base e' solida)

Questa sezione **non** introduce nuove funzionalita': esplicita i punti che rendono lo snapshot gia' “certification-grade” (in senso operativo, non regolamentare).

- **Offline-by-default + guard espliciti**: riduce il rischio di chiamate rete involontarie durante run/backtest e limita failure mode non deterministici.
- **Single source of truth (DuckDB)**: coerenza transazionale tra dati grezzi, segnali, audit, forecast e report; schema owner esplicito e migrazioni deterministiche.
- **Policy contracts / decision records nel codice**: alcune scelte sono trattate come contratti (es. timing conservativo, dividend policy) e vengono esplicitate nei transcript.
- **Transcript + fingerprinting**: ogni run e' riproducibile e ispezionabile; utile per debug, audit interno e controllo drift operativo.
- **Data Quality Gates pre-audit**: la pipeline fallisce “early” se mancano prezzi, mapping o provenance, evitando falsi positivi generati da buchi dati.
- **NEWS-ALPHA dual-lane**: separa ingest incrementale (operativo) da history lane (ricerca/backtest), mantenendo deterministicita' e dedup.

4.3.4 3.4 Aree di attenzione (priorita' di completamento, non implementate nello snapshot)

- **Execution layer assente:** oggi OBSERVER e' un decision engine (ranking + audit). Per operativita' reale serve un layer di esecuzione (paper/live) con broker API, order lifecycle e log di esecuzione.
- **Risk management implicito:** mancano limiti di posizione, sizing dinamico, controlli di concentrazione e stop/kill-switch. Va introdotto un RiskEngine "pre-trade" e post-trade.
- **Dividendi e corporate actions end-to-end:** la base dati supporta la direzione, ma la catena total-return (cashflow) va completata per strategie long-only di lungo periodo e mercati EU/UK.
- **News sentiment baseline:** l'approccio deterministico e' deliberato (auditability), ma l'accuratezza puo' essere estesa con modelli NLP piu' espressivi (sempre con vincoli di riproducibilita').

4.3.5 3.5 Scenari di utilizzo ad alta efficacia (research; richiedono validazione)

Nota: gli scenari sotto sono **ipotesi di ricerca** e richiedono validazione out-of-sample e controllo di costi/slippage. Non costituiscono promesse di performance.

- **Scenario A — News/Sentiment mean-reversion (multi-day):** uso di NEWS-ALPHA per identificare shock di sentiment e costruire trade di rientro su 1-3 giorni con regole TTL/lifecycle.
- **Scenario B — Forecast fusion (Quant + News):** fusione di segnali quant (recs) con sentiment (sentiment_cache) per ridurre drawdown e aumentare spiegabilita' del ranking.
- **Scenario C — Event-driven (GDELTA) cross-asset:** uso della lane storica per classificare eventi geopolitici/macro e costruire basket/hedge cross-asset (da considerare solo dopo aver introdotto risk controls).

4.3.6 3.6 Piano di completamento suggerito (per derivare backlog implementativo)

Phase 1 — Foundation Enhancement (chiudere il "decision engine" verso un sistema operabile)

- 1) **RiskEngine:** limiti concentrazione, max position size, sizing per volatilita', stop-loss (es. ATR-based), stress test e kill-switch.
- 2) **ExecutionGateway / OMS (paper-first):** routing ordini verso broker API (es. IBKR/Alpaca), gestione stato ordini/fill, e log completo in DB (execution_log).
- 3) **Position keeping + TCA:** libro posizioni e analisi costi effettivi (slippage/commissioni) per confrontare "forecast vs realized".
- 4) **Automazione ciclo giornaliero:** batch Windows/Linux che esegue collect (quando consentito), certify, forecast e genera output/alert (senza violare l'offline-by-default).

Phase 2 — Alpha Enhancement (incrementale, sempre auditabile)

- Upgrade del sentiment (NLP piu' ricco) mantenendo reproducibility/fingerprint.
- Ricerca e prototipazione strategie (es. pairs/event-driven) su history lane con disclosure rigorosa.

Phase 3 — Institutional features (solo se richiesto)

- Multi-portfolio support, reporting piu' strutturato, API di integrazione segnali esterni e controlli avanzati di governance del modello.

4.4 4. Applicazione come organismo (componenti e responsabilita')

Organo	Moduli principali	Funzione
Cervello (policy)	scripts/sentinel.py, env	Decide modalita' (offline/online), run_id, disciplina operativa
Memoria (Dati)	data/sentinel_alpha.db, src/db/migrate.py	DuckDB locale: schema, versioning, caching sentiment
Sistema immunitario (Gates)	src/tools/verify_*, src/core/audit_engine.py	Verifiche input, provenance, ticker mapping, no future leak
Motore (Audit)	src/core/audit_engine.py, src/core/cost_model.py, src/core/tax_model.py	Trasforma segnali in trade+equity con costi e tasse (retail realism)
Senso esterno (News)	scripts/news_alpha.py, src/news_alpha/*	Raccoglie e classifica notizie/indicatori testuali
Predizione (Forecast)	src/forecast/ranking.py	Calibra bucket firm/rating, produce forecast + stelle + ranking
Vista (UI)	app.py, pages/*	Dashboard di controllo, evidenze e monitoraggio

4.4.1 4.7 Forecast ranking (Wave 6): contratto I/O e algoritmo

Questa sezione formalizza **cosa entra** e **cosa esce** dal ranking, in modo che:

- la UI abbia un output *JSON-serIALIZZABILE* stabile;
- la pipeline resti **deterministica** (tie-break stabili);
- il comportamento “fallback” sia **disclosure-first** (retail).

Input (DuckDB):

- recs (firm, rating, sentiment_score, headline, url, date)
- audit_trades (storico trade per calibrazione)
- prices (enterability / buy_date)
- universe_membership, ticker_mappings (eligibility time-bounded)

Output (JSON-serIALIZZABILE):

- metadata: spec_name, spec_version, asof_date, universe_id, top_n
- diagnostics: candidates_total, enterable_total, dropped_missing_prices, dropped_right_censored, calibration_global_n, calibration_global_mean_return_pct
- rows: segnali con forecast_return_pct, confidence, stars + breakdown
- by_firm: aggregazioni per firm

4.4.1.1 4.7.1 Algoritmo (Step 0..6)

- 0) **As-of date:** asof_date=max(recs.date) (nel perimetro universo).
- 1) **Candidate signals:** filtra recs su asof_date, normalizza ticker, applica mapping e membership *time-bounded*.
Calcola intended_buy_date=MIN(prices.date)>signal_date. Se assente → **non-enterable** (right-censored/missing).
- 2) **Calibrazione (audit_trades):** usa signal_date<asof_date ed esclude *di default* trade con exit_reason= FALLBACK_LAST_PRICE (evita contaminazione da fallback).
- 3) **Shrinkage:** per bucket (firm,rating) “shrink” verso firm e global quando n_bucket è piccolo (robustezza retail).
- 4) **Sentiment adjustment:** clamp sentiment_score in [-1,+1], sent_adj=sentiment_score*K_SENT.
- 5) **Forecast & confidence:**
 - forecast_return_pct=shrunk_return_pct+sent_adj
 - confidence=min(1,n_bucket/N_CONF)
 - **Fallback** (se nessun trade storico): confidence=0 e forecast *solo da sentiment* con peso più alto (disclosure).
- 6) **Ranking & stars:** ranking deterministico + stelle per percentile (sintesi retail).

4.4.1.2 4.7.2 Ordinamento deterministico (tie-break) Ordina per:

- 1) forecast_return_pct desc
- 2) confidence desc
- 3) rating asc
- 4) firm asc
- 5) ticker_effective asc

4.4.2 4.8 Lifecycle Monitor: stati concettuali e mapping UI<->audit

Il Lifecycle Monitor (UI) è la **proiezione** di uno stato interno registrato nel decision ledger (audit_signal_decisions). Questa tassonomia evita ambiguità (e “magia”) e rende ogni esclusione **auditabile**.

4.4.2.1 4.8.1 Stati concettuali pre-trade (eligibility)

- **CANDIDATE:** segnale presente in recs per la data as-of.
- **ELIGIBLE:** passa normalize + ticker_mappings + universe_membership (time-bounded).
- **ENTERABLE:** esiste intended_buy_date (prezzo disponibile dopo il segnale).
- **WAITLIST:** temporaneamente non enterable per halt o gap dati (policy-dependent).
- **DROPPED:** scartato (right-censored, missing prices, fuori universo, dedup).

4.4.2.2 4.8.2 Stati trade (execution)

- **EXECUTED**: trade creato in audit_trades.
- **CLOSED**: trade chiuso con sell_date + exit_reason.
- **FALLBACK_EXIT**: uscita con prezzo fallback (**disclosure**).

4.4.2.3 4.8.3 Mapping sintetico verso stati UI

- **TRADABLE (UI)**: tipicamente ENTERABLE (pre-trade) oppure trade EXECUTED in corso (se UI mostra posizioni).
- **WAITLIST (UI)**: WAITLIST (halt/gap dati; può rientrare in TRADABLE quando torna enterable).
- **EXPIRED (UI)**: DROPPED (e/o segnali non più validi per policy/TTL, se applicato).

Nota: la UI deve sempre poter mostrare **la motivazione** (reason_code) e la catena decisionale, non solo lo “stato finale”.

4.4.2.4 4.8.4 Regole disclosure (retail)

- Ogni **skip** deve avere motivazione classificabile (skip_reason / reason_code).
- Ogni **fallback** deve essere marcato come tale e, di default, **escluso dalla calibrazione forecast** (es. DEFAULT_EXCLUDE_EXIT_REASON=FALLBACK_LAST_PRICE).

4.5 5. Fasi operative (workflow)

4.5.1 5.1 Setup e migrazione schema

- migrate: crea/aggiorna schema DuckDB e registra universi base.
- test: esegue suite test (qualita' minima prima di certify).

4.5.2 5.2 Run di audit (backtest)

- Carica segnali (recs), normalizza ticker e applica mapping time-bounded.
- Filtra per universe membership (survivorship-bias control).
- Determina buy_date/sell_date con regole conservative (T+1) e shift se dati/feasibility non consentono.
- Applica costi e (se configurata) fiscalita' retail.
- Scrive:
 - audit_runs (stato, config, fingerprint)
 - audit_trades (trade ledger)
 - audit_equity (equity curve)
 - transcript e report in reports/

4.5.3 5.3 Certify e verify

- certify: esegue run con posture piu' rigorosa (offline-by-default), produce report “certification-grade”.
- verify: rilegge un run_id e valida consistenza, provenance e regole (es. no future leak).

4.5.4 5.4 NEWS-ALPHA lane (segnali da news)

- collect: acquisisce feed (RSS o altri) quando consentito.
- run: scoring sentiment, dedup, aggregazione per ticker, output JSONL.
- Integrazione: i segnali possono alimentare recs e quindi la pipeline audit.

4.5.5 5.5 Forecast ranking (Wave 6)

- Usa lo storico audit_trades come calibrazione per bucket (firm,rating).
- Applica shrinkage verso firm/global quando il bucket e' poco popolato.
- Produce:
 - forecast_return_pct (ritorno atteso “proxy”)
 - confidence (funzione del numero osservazioni)
 - stars per percentile (ranking deterministico)

4.6 6. Principi (contratti di progettazione)

4.6.1 6.1 Offline-by-default (operational safety)

- Ogni backfill o chiamata rete deve essere esplicita (--online), altrimenti la pipeline resta offline.

- Questo riduce failure mode non deterministici e rende i risultati riproducibili.

4.6.2 6.2 Auditability first

- Ogni run ha un run_id e un code_fingerprint.
- I transcript in reports/ rendono ispezionabile la sequenza dei comandi.

4.6.3 6.3 No future leak (regola fondamentale)

- Le entrate usano un criterio conservativo: $\text{buy_date} = \text{MIN}(\text{prices.date}) \text{ WHERE } \text{prices.date} > \text{signal_date}$.
- Se i prezzi non esistono dopo il segnale (o sono censurati), la posizione e' scartata o gestita con disclosure.

4.6.4 6.4 Retail realism: costi e fiscalita'

- Cost model round-trip (commissioni/spread/slippage) applicato al ritorno lordo.
- Modello fiscale IT semplificato: CGT + loss carry ("zainetto"), e FTT/Tobin quando applicabile.

4.7 7. Ranking a stelle (Wave 6): logica e parametri

4.7.1 7.1 Obiettivo

Tradurre un insieme di segnali eterogenei in un ranking:

- leggibile (stelle 1..5)
- comparabile (forecast normalizzato)
- auditabile (deterministico, tie-breakers stabili)

4.7.2 7.2 Formula (sintesi)

Per ogni segnale:

- 1) bucket stats: media ritorni storici per (firm,rating) su audit_trades
- 2) shrinkage: bucket -> firm -> global quando n_bucket e' piccolo
- 3) sentiment adjustment: $\text{sent_adj} = \text{sentiment_score} * K_SENT$
- 4) forecast: $\text{forecast} = \text{shrunk_return} + \text{sent_adj}$
- 5) confidence: $\min(1, n_bucket / N_CONF)$
- 6) stelle: percentile su forecast (top 10% = 5 stelle, ecc.)

Parametri e fallback sono dettagliati in 007_PARAMETER_SNAPSHOT.md e nella spec specs/WAVE6_FORECAST_STARS_RANKING_SPEC.md.

4.8 8. UI (Streamlit): pagine e utilizzo

La UI e' progettata per un utente retail avanzato: visione sintetica e drill-down tecnico quando serve.

Pagine principali (vedi pages/):

- Decision Briefing: sintesi stato, risk flags, ranking.
- Pipeline Control: run/certify/verify e modalita' offline/online.
- Gates & Data Quality: mapping, membership, gap.
- Audit Runs: storico run_id e report.
- Trades & Equity: equity curve e trade ledger.
- Data Gaps & Backfill: errori e richieste provider.
- Forecasts & Ranking: stelle, percentile, confidence.
- NEWS-ALPHA: ingest/scoring, top tickers per news sentiment.
- Lifecycle Monitor: TRADABLE/WAITLIST/EXPIRED e stati traded.

4.9 9. Derivazione documenti (PDR, PDD, DATADictionary)

Questa documentazione e' stata organizzata apposta per derivare:

- **PDR**: requisiti e criteri di accettazione (vedi 002_PDR_OBSERVER.md)
- **PDD**: design e architettura (vedi 003_PDD_OBSERVER.md)
- **DataDictionary**: schema dati (vedi 004_DDT_DATADictionary.md)

La tracciabilita' Repo -> Doc e' in 005_TRACEABILITY_MATRIX.md.

4.10 10. Gap Register (cio' che manca o e' deliberatamente rinviato)

Vedi 009_GAP_REGISTER.md.

4.11 Glossario minimo

- **run_id**: identificativo esecuzione; ancora report, transcript e fingerprint.
- **universe_id**: universo di strumenti (ALL/US/EU/custom) con membership storica.
- **no future leak**: regola che impedisce uso di prezzi successivi al segnale per generare segnali stessi.
- **survivorship bias**: distorsione da universi non storicizzati (solo "sopravvissuti").
- **provenance**: origine e tracciabilita' del dato (source, fetched_at, provider).

\newpage

5 002_PDR_OBSERVER.md

6 PDR - OBSERVER (Product / Project Definition Requirements) - v1.0

Build date: 2026-01-25

6.1 1. Executive summary

OBSERVER e' un prodotto "retail advanced" che rende **verificabile** l'intero ciclo:

- 1) ingest segnali (news/analyst)
- 2) controlli di qualita' e provenance
- 3) backtest con regole conservative (no future leak)
- 4) report e ranking a stelle per decisioni operative.

La priorita' e' l'**auditability**: risultati riproducibili e spiegabili.

6.2 2. Target user e casi d'uso

6.2.1 2.1 Persona primaria: Retail avanzato "disciplinato"

- Gestisce un portafoglio equity/ETF con orizzonte multi-day.
- Vuole segnali sintetici (stelle) ma pretende trasparenza (perche' una stella? perche' escluso?).

6.2.2 2.2 Persona secondaria: Builder/quant retail

- Vuole estendere dataset, universi, gating e modelli costi/tasse.
- Necessita tool operativi, test, migrazioni e packaging sessione.

6.2.3 2.3 Casi d'uso principali

- Daily: aggiornare NEWS-ALPHA e visualizzare top ranking.
- Weekly: eseguire run/certify e consolidare report.
- Investigazione: analizzare gap dati, mapping ticker, halts, reason_code.

6.3 3. Scope (in-scope vs out-of-scope)

6.3.1 3.1 In-scope (implementato o previsto a breve)

- DuckDB come single source of truth locale (schema versionato).
- Runner CLI: migrate/test/run/certify/verify/forecast/status.
- UI Streamlit multi-pagina.
- NEWS-ALPHA offline-friendly con scoring deterministico e dedup.
- Ranking a stelle basato su storico audit + sentiment adjustment.
- Cost model e tax model (Italia) per realismo retail.

6.3.2 3.2 Out-of-scope (per questa release)

- Esecuzione live / broker integration (pianificabile come estensione: vedi Roadmap R5–R7).
- Ottimizzazione intraday / microstruttura.
- Dataset proprietari (constituents licenziati).
- Consulenza finanziaria o fiscale.

6.4 4. Requisiti funzionali (FR)

6.4.1 FR-01 Offline-by-default

Il sistema deve rimanere offline salvo abilitazione esplicita `--online` (o env equivalente).

Acceptance: `scripts/sentinel.py` certifica imposta offline se non specificato `--online`.

6.4.2 FR-02 Audit run tracciabile

Ogni run deve avere `run_id`, `transcript` e `code_fingerprint`.

Acceptance: esiste `reports/RUN_TRANSCRIPT_<run_id>.txt` e `audit_runs.code_fingerprint`.

6.4.3 FR-03 No future leak

L'entry deve usare una regola temporale conservativa e verificabile (T+1).

Acceptance: per un segnale in data D, il `buy_date` non può essere $\leq D$.

6.4.4 FR-04 Survivorship bias control

Il backtest deve filtrare strumenti usando `universe_membership` time-bounded.

Acceptance: un ticker fuori membership in data D viene escluso e registrato (decision ledger o log).

6.4.5 FR-05 NEWS-ALPHA scoring deterministico

La stessa news deve produrre lo stesso score in assenza di cambi codice.

Acceptance: caching/normalizzazione e output JSONL riproducibile.

6.4.6 FR-06 Forecast ranking deterministico

A parità di dati, l'ordinamento deve essere stabile (tie-break deterministici).

Acceptance: sort stabile e output con ordering invariato.

6.4.7 FR-07 UI di controllo

UI deve mostrare: stato DB, ultimi run, gap, ranking, lifecycle.

Acceptance: tutte le pagine in `pages/` caricano senza errori con DB presente.

6.4.8 FR-08 Risk controls (future release; non implementato nello snapshot)

Il sistema deve poter applicare controlli di rischio **pre-trade** e **post-trade** (limiti concentrazione, sizing, stop/kill-switch), separati dal ranking (separation of concerns).

Acceptance (minimo): esiste un modulo RiskEngine che, dato un ranking e un portafoglio corrente, produce una lista di ordini “ammessi/negati” con `reason_code` e parametri espliciti.

6.4.9 FR-09 Execution gateway + execution log (future release; non implementato nello snapshot)

Il sistema deve poter inviare ordini (paper-first, poi live) tramite un gateway verso broker API e registrare l'intero lifecycle ordine/fill in un log persistente.

Acceptance (minimo): esiste un endpoint `scripts/execute.py` (o equivalente) che legge un ranking “latest”, crea ordini e scrive un `execution_log` in DB (almeno: `order_id`, `ticker`, `side`, `qty`, `submitted_at`, `status`, `fill_px`, `fees`).

6.4.10 FR-10 Model monitoring (future release; non implementato nello snapshot)

Il sistema deve poter monitorare drift del modello confrontando forecast vs realized, producendo alert quando metriche chiave degradano (es. IC, hit-rate, slippage).

Acceptance (minimo): esiste un report/summary che calcola metriche su finestre rolling e alza un flag nel run report quando supera soglie configurate.

6.5 5. Requisiti non funzionali (NFR)

- **NFR-01 Riproducibilit :** la stessa run su stesso snapshot produce stessi artefatti (salvo timestamp).
- **NFR-02 Robustezza:** errori provider non devono corrompere DB; devono essere loggati in data_gaps.
- **NFR-03 Trasparenza:** ogni decisione “skip” deve essere motivabile (skip_reason/reason_code).
- **NFR-04 Usabilit  retail:** output leggibile (stelle, confidence) con drill-down tecnico.
- **NFR-05 Sicurezza:** niente segreti hardcoded; chiavi API via env.

6.6 6. Criteri di accettazione (Definition of Done)

- Coverage 100% nella 005_TRACEABILITY_MATRIX.
- Zero riferimenti a componenti inesistenti (no “voci fantasma”).
- PDF descrittivo senza placeholder e senza difetti di layout.
- py-mpytest passa nello snapshot.
- scripts/sentinel.pystatus e scripts/news_alpha.pystatus funzionano (DB presente).

6.7 7. Roadmap suggerita (retail + rigore monetario/finanziario)

- R1: “Macro context layer” (tassi, inflazione, banche centrali) come fattore di rischio/regime.
- R2: FX translation e multi-currency (utile per titoli EU/US in portafogli EUR).
- R3: Dividendi total return (cashflow) e corporate actions.
- R4: Execution realism dinamico (spread/liquidit ) con parametri retail.
- R5: RiskEngine (position sizing, limiti concentrazione, stop/kill-switch) come gate pre-trade.
- R6: ExecutionGateway/OMS paper-first + log ordini/fill (Transaction Cost Analysis inclusa).
- R7: Automazione ciclo giornaliero (collect -> certify -> forecast -> alert/execute) rispettando l’offline-by-default.
- R8: Model governance: monitor forecast vs realized (drift) e metriche di stabilit  (IC/hit-rate).

6.8 8. Rischi e mitigazioni

- **Dati incompleti** -> mitigazione: data_gaps, backfill controllato, disclosure.
- **Leakage involontario** -> mitigazione: verify, regole T+1, test dedicati.
- **Overfitting su storico** -> mitigazione: shrinkage, confidence, disclosure (non “promesse” di performance).
- **Aspettative utente** -> mitigazione: linguaggio retail, disclaimer, spiegabilit  per ogni trade.

\newpage

7 003_PDD_OBSERVER.md

8 PDD - OBSERVER (Product / Project Design Document) - v1.0

Build date: 2026-01-25

8.1 1. Design intent

Progettare un sistema retail avanzato con standard “institutional-grade” su:

- tracciabilit  (run_id, transcript, fingerprint)
- conservative timing (T+1, no future leak)
- controlli su universi, ticker mapping e halts
- output leggibile (stelle, confidence) e UI operativa.

8.2 2. Architettura (layer)

8.2.1 2.1 Layer operativi (entrypoint)

- scripts/sentinel.py: runner principale (migrate/test/run/certify/verify/forecast/status)
- scripts/news_alpha.py: runner NEWS-ALPHA (collect/run/status)
- scripts/ops_run_session.py: workflow run->pack->certify->pack
- scripts/pack_session.py: packaging deterministico dei report
- scripts/setup.py: bootstrap/repair cross-platform

8.2.2 2.2 Layer applicativo (src/)

- src/core/*: audit engine, lifecycle, cost/tax, sentiment deterministic
- src/db/*: schema owner, connection, audit_store
- src/news_alpha/*: ingest + scoring + dedup + db integration
- src/forecast/*: calibrazione e ranking a stelle
- src/tools/*: verify e strumenti diagnostici

8.2.3 2.3 Layer presentazione (UI)

- app.py + pages/*.py: dashboard Streamlit modulare.

8.3 3. Dataflow (end-to-end)

- 1) **Input segnali:** recs (da feed, news-alpha o import)
- 2) **Normalization & mapping:** ticker normalizzato + ticker_mappings time-bounded
- 3) **Universe filter:** universe_membership (survivorship-bias control)
- 4) **Feasibility & timing:** prices e regola $\text{MIN}(\text{date}) > \text{signal_date}$
- 5) **Trade simulation:** costi + tasse + halts -> audit_trades
- 6) **Equity curve:** audit_equity
- 7) **Report:** reports/ + transcript
- 8) **Forecast:** forecast/ranking.py produce ranking+stelle
- 9) **UI:** visualizza stato, evidenze e decision brief.

8.4 4. Data layer (DuckDB)

- Schema definito in src/db/migrate.py (owner canonico).
- Database locale: data/sentinel_alpha.db.
- sentiment_cache consente scoring deterministico e ripetibile (auditability).

Riferimento: 004_DDT_DATADictionary.md.

8.5 5. Audit engine: regole chiave

8.5.1 5.1 No future leak (T+1)

- buy_date = minimo prezzo con data strettamente successiva al segnale.
- evitare qualsiasi uso di close della stessa giornata del segnale.

8.5.2 5.2 Execution halts

- ticker_halts e market_halts bloccano o shiftano execution.
- reason codes vengono persistiti (halt_reason).

8.5.3 5.3 Retail realism

- Cost model: round-trip cost pct (proxy spread+commissioni+slippage).
- Tax model (Italia): CGT + loss carry; FTT/Tobin come costo addizionale quando applicabile.

8.6 6. Forecast & stars ranking (Wave 6)

- Calibrazione su audit_trades storici.
- Shrinkage per robustezza su piccoli campioni (retail tipicamente ha poco storico).
- Confidence esplicita per evitare “falsa precisione”.
- Ordinamento deterministico con tie-break stabili.

Spec: specs/WAVE6_FORECAST_STARS_RANKING_SPEC.md

Parametri: 007_PARAMETER_SNAPSHOT.md.

8.7 7. Operabilita' e packaging

- Transcript run/certify in reports/.
- pack_session.py crea cartelle deterministiche per archiviazione e condivisione.
- ops_run_session.py codifica un flusso standard (riduce errori operativi).

8.8 8. Estensioni future (coerenti con trading monetario/finanziario)

- Layer macro: policy rate, inflazione, curve, risk-off/risk-on -> adjustment risk_scalar.
- FX translation: PnL in valuta base (EUR) con serie FX e conversioni.
- Total return: dividendi e corporate actions integrate nella equity curve.
- Execution realism: spread dinamico e limitazioni liquidity retail.

8.8.1 8.1 RiskEngine (pre-trade + post-trade) — proposta

Obiettivo: separare chiaramente **ranking** (alpha) da **ammissibilita' operativa** (risk).

- Input: ranking “latest”, portafoglio corrente, vincoli (max position size, max sector exposure, risk_scalar, stop policy, liquidity cap).
- Output: lista ordini proposta + lista ordini negati con reason_code e parametri espliciti.
- Integrazione: hook immediatamente **prima** dell'esecuzione (paper/live) e reportistica post-trade.

Tabelle suggerite (non presenti nello snapshot):

- positions (as-of, qty, avg_cost, realized/unrealized)
- risk_flags (run_id, ticker, flag_type, severity, message)

8.8.2 8.2 ExecutionGateway / OMS (paper-first) — proposta

Obiettivo: chiudere l'anello “decision -> ordine” mantenendo auditability e offline-by-default.

- Entry point suggerito: scripts/execute.py (paper trading di default; live richiede flag esplicito e secret via env).
- Gateway: adapter verso broker API (es. IBKR/Alpaca) con gestione lifecycle (submit, ack, partial fill, fill, cancel, reject).
- Persistenza: execution_log in DuckDB per tracciare ogni evento (ordini e fill) con timestamp e fingerprint.

Tabelle suggerite (non presenti nello snapshot):

- execution_log (order_id, run_id, ticker, side, qty, submitted_at, status, fill_px, fees, venue, raw_payload_hash)
- orders_intent (rank_snapshot_id, constraints_applied, risk_version)

8.8.3 8.3 Transaction Cost Analysis + Model Monitoring — proposta

Obiettivo: misurare lo scarto tra “simulato” e “realizzato” e rilevare drift del modello.

- TCA: slippage, commissioni, spread implicito per bucket di liquidita'/market cap.
- Monitoring: forecast vs realized su finestre rolling; metriche (IC, hit-rate, drawdown, turnover, cost drag).
- Output: alert/flag nei report (o in UI) quando soglie degradano.

8.9 9. Considerazioni di rischio e compliance (retail)

- Il sistema non e' consulenza finanziaria; i report sono strumenti di analisi.
- La scelta di usare modelli deterministici e conservative timing e' deliberata per ridurre bias.
- Ogni output deve includere disclosure dei limiti dati e dei fallback.

\newpage

9 004_DDT_DATADictionary.md

10 DATADictionary (DuckDB) - v1.0

Build date: 2026-01-24

10.1 Panoramica

OBSERVER / SENTINEL-ALPHA utilizza **DuckDB** come single source of truth locale.

Il modello dati e' progettato per:

- riproducibilita' e auditability (run_id, code_fingerprint, transcript)
- backtest "certification-grade" (no survivorship bias, ticker mappings time-bounded, halts)
- realismo retail (costi transazione, tasse IT, dividendi come estensione)

10.1.1 Convenzioni generali

- DATE e' usato per la logica di sessione (daily).
- TIMESTAMPTZ/TIMESTAMP sono usati per logging e provenance. Quando possibile, usare UTC.
- Le chiavi sono implementate come PRIMARYKEY o UNIQUE (in alcuni casi DuckDB materializza la chiave come UNIQUE).

10.1.2 Relazioni logiche (alto livello)

- audit_runs 1..N audit_trades / audit_equity / audit_signal_decisions / data_gaps
- metadata 1..N prices / dividends
- recs alimenta audit_signal_decisions e, indirettamente, audit_trades

10.2 metadata

Anagrafica strumenti: ticker canonico e attributi di mercato/settore, inclusi flag fiscali (Tobin/FTT) e simboli provider.

10.2.1 Colonne

Colonna	Tipo	NULL?	Semantica
ticker	VARCHAR	NO	Ticker canonico interno (chiave primaria).
sector	VARCHAR	YES	Settore economico (granularita' libera).
market	VARCHAR	YES	Etichetta mercato/regione (es. US, EU, ITALY).
currency	VARCHAR	YES	Valuta di quotazione (ISO-4217, es. USD, EUR).
is_tobin_tax	BOOLEAN	YES	Flag: applica Tobin/FTT (Italia) come costo transazione addizionale.
yf_symbol	VARCHAR	YES	Simbolo provider yfinance (se diverso dal ticker canonico).
stooq_symbol	VARCHAR	YES	Simbolo provider stooq (se diverso dal ticker canonico).
instrument_type	VARCHAR	YES	Classe strumento (EQUITY, ETF, DERIVATIVE, ...).
ftt_rate	DOUBLE	YES	Aliquota FTT come frazione del notional (es. 0.001 = 0.10%).

10.2.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(ticker)

10.3 prices

Prezzi giornalieri (barre daily): close (obbligatorio) + open opzionale, con provenance e timestamp di inserimento.

10.3.1 Colonne

Colonna	Tipo	NULL?	Semantica
date	DATE	YES	Data sessione (daily).
ticker	VARCHAR	YES	Ticker canonico.
price	FLOAT	YES	Prezzo di chiusura (close).
open_price	FLOAT	YES	Prezzo di apertura (open), se disponibile.
source	VARCHAR	YES	Provenienza riga (legacy/yfinance/stooq/...).
fetch_at	TIMESTAMP	YES	Timestamp inserimento/aggiornamento riga.

10.3.2 Vincoli (DuckDB)

- UNIQUE: UNIQUE(date, ticker)

10.4 dividends

Eventi dividendo per realismo retail (ex-date e pay-date). Nello snapshot lo schema e' presente; l'applicazione cashflow e' incrementale.

10.4.1 Colonne

Colonna	Tipo	NULL?	Semantica
ticker	VARCHAR	NO	Ticker canonico.
ex_date	DATE	NO	Data ex-dividend.
pay_date	DATE	YES	Data pagamento.
amount	DOUBLE	YES	Importo dividendo per share/unit.
currency	VARCHAR	YES	Valuta importo.
source	VARCHAR	YES	Provenienza dato.
fetch_at	TIMESTAMP	YES	Timestamp inserimento.

10.4.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(ticker, ex_date)

10.5 recs

Store segnali/raccomandazioni (news/analyst): per data di pubblicazione, ticker, firm e rating, con sentiment score e metadati.

10.5.1 Colonne

Colonna	Tipo	NULL?	Semantica
date	DATE	YES	Data pubblicazione segnale (legacy compatible).
ticker	VARCHAR	YES	Ticker del segnale (puo' essere non canonico: normalizzato/mappato a runtime).
firm	VARCHAR	YES	Fonte/firm (es. banca d'affari, provider news).
rating	VARCHAR	YES	Rating discreto (BUY/HOLD/DOWNGRADE o analogo).
sentiment_score	DOUBLE	YES	Sentiment in [-1,+1] (deterministico, cacheabile).
headline	VARCHAR	YES	Titolo/news headline associata al segnale.
source_url	VARCHAR	YES	URL sorgente (se disponibile).
universe_id	VARCHAR	YES	Universe associato al segnale (ALL/US/EU o custom).
published_at	TIMESTAMP	YES	Timestamp pubblicazione (se disponibile).

10.5.2 Vincoli (DuckDB)

- UNIQUE: UNIQUE(date, ticker, firm)

10.6 momentum_rankings

Ranking momentum giornaliero per ticker (feature/gate) con rendimento periodo e segnali discreti.

10.6.1 Colonne

Colonna	Tipo	NULL?	Semantica
date	DATE	NO	Data calcolo ranking.
ticker	VARCHAR	NO	Ticker canonico.
monthly_return	FLOAT	YES	
rank_pos	INTEGER	YES	
signal	VARCHAR	YES	Segnale discreto (es. LONG/SHORT/HOLD).

10.6.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(date, ticker)

10.7 universes

Catalogo universi (es. ALL/US/EU) per filtrare segnali e controllare survivorship bias.

10.7.1 Colonne

Colonna	Tipo	NULL?	Semantica
universe_id	VARCHAR	NO	ID universe (chiave primaria).
name	VARCHAR	YES	Nome descrittivo.
market	VARCHAR	YES	Mercato/regione prevalente (US/EU/MULTI...).
description	VARCHAR	YES	Descrizione testuale.

10.7.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(universe_id)

10.8 universe_membership

Membership storica (ticker in universo) con intervalli temporali: essenziale per backtest senza survivorship bias.

10.8.1 Colonne

Colonna	Tipo	NULL?	Semantica
universe_id	VARCHAR	NO	ID universe.
ticker	VARCHAR	NO	Ticker canonico.
start_date	DATE	NO	Inizio validita' membership (inclusivo).
end_date	DATE	YES	Fine validita' (inclusivo, NULL=open-ended).
source	VARCHAR	YES	Fonte dataset membership.
notes	VARCHAR	YES	Note/annotazioni.

10.8.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- NOT NULL: NOT NULL
- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(universe_id, ticker, start_date)

10.9 ticker_mappings

Mappature alias->canonico time-bounded (corporate actions, cambio simbolo) per evitare mismatch ticker.

10.9.1 Colonne

Colonna	Tipo	NULL?	Semantica
alias_ticker	VARCHAR	NO	Simbolo alias (come appare nel dato grezzo).
canonical_ticker	VARCHAR	YES	Simbolo canonico interno.
start_date	DATE	NO	Inizio validita' mapping.
end_date	DATE	YES	Fine validita' mapping (NULL=open-ended).
source	VARCHAR	YES	Fonte mapping (manual, provider, corporate actions...).
notes	VARCHAR	YES	Note.

10.9.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(alias_ticker, start_date)

10.10 ticker_halts

Halt su singolo ticker: finestra temporale di non-tradabilita' e reason/provenance.

10.10.1 Colonne

Colonna	Tipo	NULL?	Semantica
ticker	VARCHAR	NO	Ticker canonico.
start_date	DATE	NO	Inizio halt.
end_date	DATE	YES	Fine halt (NULL=open-ended).
reason	VARCHAR	YES	Motivo halt (sospensione, delisting, market maker, ecc.).
source	VARCHAR	YES	Fonte informazione.

10.10.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(ticker, start_date)

10.11 market_halts

Halt di mercato/paese (es. circuit breaker, chiusure straordinarie) per execution feasibility.

10.11.1 Colonne

Colonna	Tipo	NULL?	Semantica
market	VARCHAR	NO	Mercato/regione (es. US, EU).
start_date	DATE	NO	Inizio halt.
end_date	DATE	YES	Fine halt.
reason	VARCHAR	YES	Motivo halt (circuit breaker, chiusure, ecc.).
source	VARCHAR	YES	Fonte informazione.

10.11.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(market, start_date)

10.12 sentiment_cache

Cache locale deterministica del sentiment (testo normalizzato + hash) per ripetibilit  e audit.

10.12.1 Colonne

Colonna	Tipo	NULL?	Semantica
text_hash	VARCHAR	NO	Hash del testo normalizzato (chiave primaria).
text	VARCHAR	YES	Testo normalizzato (troncato a 2000 char).
score	DOUBLE	YES	Sentiment score in [-1,+1].
model	VARCHAR	YES	Modello usato (vader/lexicon).
computed_at	TIMESTAMP	YES	Timestamp calcolo (UTC).

10.12.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(text_hash)

10.13 data_gaps

Log operativo di gap/backfill (prezzi/news): include intervalli richiesti, esito, righe inserite e reason_code standardizzato.

10.13.1 Colonne

Colonna	Tipo	NULL?	Semantica
kind	VARCHAR	YES	Tipo gap (prices/news/...).
ticker	VARCHAR	YES	Ticker target (se applicabile).
start_date	DATE	YES	Inizio finestra da ottenere (normalizzata).
end_date	DATE	YES	Fine finestra da ottenere.
requested_at	TIMESTAMP	YES	Timestamp richiesta.
status	VARCHAR	YES	SUCCESS/FAILED/SKIPPED.
provider	VARCHAR	YES	Provider (yfinance/stooq/gdelt/...).
message	VARCHAR	YES	Messaggio sintetico esito.
rows_inserted	INTEGER	YES	Numero righe inserite.
error	VARCHAR	YES	Errore raw (se presente).
duration_ms	INTEGER	YES	Durata operazione.
run_id	VARCHAR	YES	Run che ha richiesto il backfill (se presente).
rows_upserted	INTEGER	YES	Numero righe inserite + aggiornate.
reason_code	VARCHAR	YES	Reason code standardizzato (per KPI data quality).
requested_start_date	DATE	YES	Inizio richiesto originario (prima di clamp/parse).
requested_end_date	DATE	YES	Fine richiesta originaria.
obtained_start_date	DATE	YES	Min date effettivamente ottenuta.
obtained_end_date	DATE	YES	Max date effettivamente ottenuta.

10.13.2 Vincoli (DuckDB)

- (nessun vincolo dichiarato)

10.14 audit_runs

Registro run di audit/certificazione: configurazione, universi, fingerprint del codice e stato esecuzione.

10.14.1 Colonne

Colonna	Tipo	NULL?	Semantica
run_id	VARCHAR	NO	Identificativo run (chiave primaria, deterministico/UUID-like).
started_at	TIMESTAMP WITH TIME ZONE	YES	Timestamp inizio run (UTC consigliato).
finished_at	TIMESTAMP WITH TIME ZONE	YES	Timestamp fine run.
status	VARCHAR	YES	Stato: RUNNING/SUCCESS/FAILED.
universe_id	VARCHAR	YES	Universe testato.
holding_period_sessions	INTEGER	YES	Holding period in sessioni (T+N, default definito nella pipeline).
config_json	VARCHAR	YES	Configurazione run serializzata (string JSON).
code_fingerprint	VARCHAR	YES	Fingerprint del codice (hash) per audit.
notes	VARCHAR	YES	Note manuali / annotazioni.
error	VARCHAR	YES	Messaggio errore (se FAILED).

10.14.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(run_id)

10.15 audit_signal_decisions

Decision ledger opzionale: traccia segnali eseguiti o scartati (dedup, halt, skip) e date ‘intese’ vs effettive.

10.15.1 Colonne

Colonna	Tipo	NULL?	Semantica
decision_id	VARCHAR	NO	ID decisione (chiave primaria).
run_id	VARCHAR	YES	ID run.
signal_date	DATE	YES	Data segnale.
ticker_original	VARCHAR	YES	Ticker originale.
ticker	VARCHAR	YES	Ticker canonico effettivo.
firm	VARCHAR	YES	Fonte segnale.
rating	VARCHAR	YES	Rating.
universe_id	VARCHAR	YES	Universe.
intended_buy_date	DATE	YES	Buy date intesa (prima di shift).
buy_date	DATE	YES	Buy date effettiva.
exec_shift_sessions	INTEGER	YES	Shift buy.
intended_sell_date	DATE	YES	Sell date intesa.
sell_date	DATE	YES	Sell date effettiva.
exit_shift_sessions	INTEGER	YES	Shift sell.
decision	VARCHAR	YES	EXECUTED / SKIPPED / DROPPED_DEDUP.
skip_reason	VARCHAR	YES	Motivo skip (missing price, halt, policy).
halt_reason	VARCHAR	YES	Motivo halt (ticker/market).
created_at	TIMESTAMP WITH TIME ZONE	YES	Timestamp creazione record.

10.15.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(decision_id)

10.16 audit_trades

Trade ledger per run: ogni trade deriva da un segnale, include shift esecutivi, motivi di uscita e performance lorda/netta.

10.16.1 Colonne

Colonna	Tipo	NULL?	Semantica
trade_id	VARCHAR	NO	ID trade (chiave primaria).
run_id	VARCHAR	YES	FK logica verso audit_runs.run_id.
signal_date	DATE	YES	Data segnale originale (giorno di pubblicazione).
buy_date	DATE	YES	Data entrata effettiva (con shift e vincoli).
sell_date	DATE	YES	Data uscita effettiva.
exit_reason	VARCHAR	YES	Motivo uscita (TARGET/HOLDING_PERIOD/HALT/FALLBACK_LAST_PRICE...).
exit_is_fallback	BOOLEAN	YES	Flag: uscita fallback (prezzo last known).
ticker	VARCHAR	YES	Ticker effettivo canonico usato per prezzi e membership.
firm	VARCHAR	YES	Fonte segnale.
rating	VARCHAR	YES	Rating del segnale.
market	VARCHAR	YES	Mercato (derivato da metadata o regole).
sector	VARCHAR	YES	Settore (derivato da metadata).
mom_status	VARCHAR	YES	Esito gate momentum (se applicato).
risk_vol	DOUBLE	YES	Volatilità/proxy rischio usato nella gestione posizione (se presente).
is_tobin_tax	BOOLEAN	YES	Flag FTT applicata.
sentiment_score	DOUBLE	YES	Sentiment associato al segnale.
buy_price	DOUBLE	YES	Prezzo entrata (close o regola scelta).
sell_price	DOUBLE	YES	Prezzo uscita.
gross_return_pct	DOUBLE	YES	Rendimento lordo (%) pre-costi e pre-tasse.
cost_pct	DOUBLE	YES	Costo transazione totale (%) usato nel modello.
net_return_pct	DOUBLE	YES	Rendimento netto (%) dopo costi (tasse tracciate altrove).
trade_score	DOUBLE	YES	Score del trade (se applicato dal sistema).
universe_id	VARCHAR	YES	Universe in cui il trade e' stato eseguito/valutato.
ticker_original	VARCHAR	YES	Ticker originale del segnale (prima di normalize/mapping).
instrument_type	VARCHAR	YES	Classe strumento.
ftt_pct	DOUBLE	YES	FTT effettiva usata come percentuale del notional.
exec_shift_sessions	INTEGER	YES	Shift entrata (sessioni) per vincoli T+1 / prezzi mancanti / halt.
exit_shift_sessions	INTEGER	YES	Shift uscita (sessioni).
halt_reason	VARCHAR	YES	Motivo di halt (ticker/market) se ha impattato l'esecuzione.

10.16.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(trade_id)

10.17 audit_equity

Equity curve giornaliera per run: equity, cash, investito, numero posizioni e tasse pagate.

10.17.1 Colonne

Colonna	Tipo	NULL?	Semantica
run_id	VARCHAR	NO	ID run.
date	DATE	NO	Data (daily).
equity	DOUBLE	YES	Equity totale (cash + posizioni mark-to-market).
cash	DOUBLE	YES	Cassa disponibile.
invested	DOUBLE	YES	Notional investito (approssimazione).
positions	INTEGER	YES	Numero posizioni aperte.
tax_paid	DOUBLE	YES	Tasse pagate cumulative o per giorno (dipende dalla pipeline).
executed_trades	INTEGER	YES	Numero trade eseguiti nel giorno.
closed_trades	INTEGER	YES	Numero trade chiusi nel giorno.

10.17.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(run_id, date)

\newpage

11 005_TRACEABILITY_MATRIX.md

12 Traceability Matrix (Repo -> Documenti) - v1.1

Build date: 2026-01-24

12.1 Scopo

Questa matrice garantisce che:

- nessun componente rilevante del repository sia “orfano” (non descritto)
- ogni sezione documentale rimandi a componenti reali
- la completezza sia verificabile tramite comandi e artefatti

12.2 Matrice

Componente (path)	Classe	Stato	Documentazione (ref)	Verifica rapida
app.py	Streamlit app entrypoint (routing UI pages)	IMPLEMENTATO	001§8; 003§6 (UI)	py-mstreamlitrunapp.py
main.py	CLI orchestrator (batch)	IMPLEMENTATO	003§5 (Orchestrazione)	pymain.py--help
scripts/news_alpha.py	NEWS-ALPHA operator runner.	IMPLEMENTATO	001§5; specs/NEWS_ALPHA_SPEC; 008	pyscripts/news_alpha.py--help
scripts/ops_reset.py	Operational reset utility (NEWS-ALPHA / SENTINEL-ALPHA).	IMPLEMENTATO	001§5/Appendice A; 003§7 (Ops); 008	pyscripts/ops_reset.py--help
scripts/ops_run_session.py	Run -> pack -> certify -> pack (DB-first auto-run-id).	IMPLEMENTATO	001§5; 008 (session workflow)	pyscripts/ops_run_session.py--help
scripts/pack_session.py	Pack reports into deterministic session folders (no symlinks).	IMPLEMENTATO	008 (packaging); 003§7	pyscripts/pack_session.py--help
scripts/patch_persist_equity.py	Utility operativa	IMPLEMENTATO	001§5/Appendice A; 003§7 (Ops); 008	pyscripts/patch_persist_equity.py--help
scripts/sentinel.py	SENTINEL-ALPHA one-command runner.	IMPLEMENTATO	001§5-6; specs/SENTINEL_RUNNER_SPEC; 008	pyscripts/sentinel.py--help
scripts/setup.py	SENTINEL-ALPHA bootstrap / repair entrypoint (cross-platform).	IMPLEMENTATO	008 (bootstrap); 003§7	pyscripts/setup.py--help
src/db/migrate.py	Schema owner + migrations DuckDB	IMPLEMENTATO	004; 003§4 (Data layer)	py-msrc.db.migrate--dbdata/sentinel_alpha.db
src/db/audit_store.py	Persistence layer per audit artifacts	IMPLEMENTATO	003§4-5; 008	(invocato dai runner)
src/core/audit_engine.py	Audit engine (no future leak, trades, equity)	IMPLEMENTATO	001§6; specs/AUDIT_LIFECYCLE_SPEC; 003§5	pyscripts/sentinel.pyrun--help
src/core/alert_lifecycle.py	Lifecycle monitor (TRADABLE/WAITLIST/EXPIRED + traded states)	IMPLEMENTATO	001§6; specs/AUDIT_LIFECYCLE_SPEC	py-msrc.tools.alert_lifecycle--help
src/forecast/ranking.py	Forecast ranking a stelle (Wave 6)	IMPLEMENTATO	001§7; specs/WAVE6_FORECAST...; 007	py-msrc.tools.forecast_rankings--help
src/news_alpha/run.py	News ingestion + scoring + dedup + aggregation	IMPLEMENTATO	001§5; specs/NEWS_ALPHA_SPEC; 007	pyscripts/news_alpha.pyrun--help
src/core/cost_model.py	Retail cost model (round-trip cost)	IMPLEMENTATO	003§8 (Costi) ; 007	(usato in audit_engine)
src/core/tax_model.py	Modello fiscale IT (CGT + loss carry)	IMPLEMENTATO	003§8 (Tasse) ; 007	(usato in audit_engine)
pages/00_Decision_Briefing.py	Pagina Streamlit	IMPLEMENTATO	001§8; 003§6	(UI)
pages/01_Pipeline_Control.py	Pagina Streamlit	IMPLEMENTATO	001§8; 003§6	(UI)
pages/02_Gates_Data_Quality.py	Pagina Streamlit	IMPLEMENTATO	001§8; 003§6	(UI)
pages/03_Audit_Runs.py	Pagina Streamlit	IMPLEMENTATO	001§8; 003§6	(UI)

12.3 Regola di accettazione (operativa)

- **Coverage 100% sui componenti rilevanti:** runner, pipeline, data layer, UI, strumenti.
- **Zero voci fantasma:** nulla e' descritto come “presente” se non esiste nello snapshot.
- **Verifiche ripetibili:** per ogni runner esiste almeno un comando --help o status che conferma l'installazione.

\newpage

13 006__REPO__BOM.md

14 Repository BOM (Bill of Materials) - v1.1

Build date: 2026-01-24

14.1 Sintesi

Questo documento elenca i componenti del repository presenti nello snapshot incluso nello ZIP, con focus su:

- entrypoint e runner CLI
- UI Streamlit (pagine)
- moduli core (audit, lifecycle, forecasting, news-alpha)
- configurazione locale e dati (DuckDB)

14.2 Struttura ad alto livello

- app.py: entrypoint Streamlit (UI)
- main.py: entrypoint CLI / orchestrazione (batch)
- scripts/: runner operativi (SENTINEL e NEWS-ALPHA)
- src/: logica applicativa e librerie interne
- data/: DuckDB locale e cache (offline-friendly)
- reports/: artefatti di audit, transcript e report (riproducibilita')
- runs/: staging run_id e output di sessione
- config/: env e helper (PowerShell / .env)

14.3 Runner e utility operative (scripts/)

File	Ruolo
scripts/news_alpha.py	NEWS-ALPHA operator runner.
scripts/ops_reset.py	Operational reset utility (NEWS-ALPHA / SENTINEL-ALPHA).
scripts/ops_run_session.py	Run -> pack -> certify -> pack (DB-first auto-run-id).
scripts/pack_session.py	Pack reports into deterministic session folders (no symlinks).
scripts/patch_persist_equity.py	
scripts/sentinel.py	SENTINEL-ALPHA one-command runner.
scripts/setup.py	SENTINEL-ALPHA bootstrap / repair entrypoint (cross-platform).

14.4 UI Streamlit (pages/)

File	Ruolo
pages/00_Decision_Briefing.py	Pagina Streamlit (UI)
pages/01_Pipeline_Control.py	Pagina Streamlit (UI)
pages/02_Gates_Data_Quality.py	Pagina Streamlit (UI)
pages/03_Audit_Runs.py	Pagina Streamlit (UI)
pages/04_Trades_Equity.py	Pagina Streamlit (UI)
pages/05_Data_Gaps_Backfill.py	Pagina Streamlit (UI)
pages/06_Forecasts_Ranking.py	Pagina Streamlit (UI)
pages/07_NEWS_ALPHA.py	Pagina Streamlit (UI)
pages/08_Lifecycle_Monitor.py	Pagina Streamlit (UI)

14.5 Moduli applicativi principali (src/*.py)

File	Ruolo
src/init.py	Modulo applicativo (orchestrazione/servizi)
src/analyst_auditor.py	Modulo applicativo (orchestrazione/servizi)
src/intelligence_engine.py	Modulo applicativo (orchestrazione/servizi)
src/morning_bulletin.py	Modulo applicativo (orchestrazione/servizi)
src/performance_analyzer.py	Modulo applicativo (orchestrazione/servizi)
src/report_generator.py	Modulo applicativo (orchestrazione/servizi)
src/sentinel_alpha.py	Modulo applicativo (orchestrazione/servizi)

14.6 Package interni

Package	Contenuto
src/core/	7 moduli Python
src/db/	4 moduli Python
src/forecast/	2 moduli Python
src/news_alpha/	8 moduli Python
src/tools/	11 moduli Python
src/data/	2 moduli Python

14.7 Note operative (retail)

- Il repository include un database locale DuckDB (data/sentinel_alpha.db).
In un contesto retail e' corretto considerarlo un "single-user local store" per riproducibilita' e audit.
- La cartella reports/ include artefatti storici. Per release "pulite" si puo' allegare solo l'ultimo *evidence pack*.

\newpage

15 007_PARAMETER_SNAPSHOT.md

16 Parameter Snapshot - v1.1

Build date: 2026-01-24

16.1 Scopo

Questa sezione riduce il rischio principale delle documentazioni "lunghe": la divergenza tra descrizione e implementazione.

I valori sotto sono estratti dal codice nello snapshot incluso nello ZIP (non sono "ri-scritti" manualmente).

Area	Parametro	Valore (dal codice)
Forecast ranking spec	SPEC_NAME	WAVE6_FORECAST_STARS_RANKING_SPEC.md
Forecast ranking spec	SPEC_VERSION	v0.1
Forecast ranking	N_MIN (min osservazioni bucket)	20
Forecast ranking	N_CONF (normalizzazione confidenza)	60
Forecast ranking	W_GLOBAL (peso minimo prior globale)	0.25
Forecast ranking	K_SENT (peso sentiment su forecast)	0.20
Forecast ranking	Fallback (global_n==0) sentiment multiplier	0.50
NEWS-ALPHA rating	Soglia BUY	0.20
NEWS-ALPHA rating	Soglia DOWNGRADE	-0.20
Cost model	Round-trip cost pct	0.0075
Tax model (IT)	Capital gains tax rate	0.26
Local sentiment cache	Hash algorithm	sha1
Local sentiment cache	Lexicon size POS/NEG	28/27
DuckDB schema	Schema version (migrate.py)	2.1.0
DuckDB schema	Last updated	2026-01-16

16.2 Note interpretative (retail)

- Round-tripcostpct e' un proxy conservativo (commissioni + spread + slippage) espresso come frazione di notional.
- La tassazione italiana e' modellata in modo semplificato (non e' consulenza fiscale).
- Il fallback del ranking (assenza storico audit_trades) usa *solo* sentiment, con peso piu' alto (0.50) per evitare forecast "piatti" a 0.

\newpage

17 008_EVIDENCE_PACK.md

18 Evidence Pack - v1.1

Build date: 2026-01-24

18.1 Obiettivo

Fornire una sequenza operativa **riproducibile** per:

- installare dipendenze
- migrare/validare schema DuckDB
- eseguire run/certify/verify
- produrre forecast ranking
- eseguire NEWS-ALPHA
- avviare UI

Include inoltre convenzioni su artefatti e naming.

18.2 1) Setup ambiente

18.2.1 Windows (PowerShell)

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
py -m pip install -r requirements.txt
```

18.2.2 Linux/macOS (bash)

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

18.3 2) Schema e stato DB (DuckDB)

```
py scripts/sentinel.py migrate --db data/sentinel_alpha.db
py scripts/sentinel.py status --db data/sentinel_alpha.db
```

Artefatto atteso: tabelle create/aggiornate (vedi 004_DDT_DATADictionary.md).

18.4 3) Test suite

```
py -m pytest
# oppure via runner
py scripts/sentinel.py test
```

Gate consigliato: nessun test rosso prima di certify.

18.5 4) Run / Certify / Verify (SENTINEL-ALPHA)

18.5.1 4.0 Postura offline/online (precedenza + contract operativo)

Precedenza posture:

- 1) --online
- 2) --offline / --no-backfill
- 3) certify = default offline (offline-by-default)

Variabili ambiente rilevanti:

- SENTINEL_ALLOW_ONLINE_BACKFILL
- SENTINEL_OFFLINE
- SENTINEL_PRICE_PROVIDER_ORDER
- SENTINEL_DISABLE_YFINANCE (default 1)

18.5.2 4.1 Run (operativo)

```
py scripts/sentinel.py run
```

- Genera o preserva SENTINEL_RUN_ID
- Scrive transcript: reports/RUN_TRANSCRIPT_<run_id>.txt

18.5.3 4.2 Certify (postura audit - offline di default)

```
py scripts/sentinel.py certify
```

- Offline-by-default: per consentire rete serve --online e policy/env coerenti.
- Transcript: reports/CERTIFY_TRANSCRIPT_<run_id>.txt

18.5.4 4.3 Verify (post-run)

```
py scripts/sentinel.py verify --run-id <run_id>
```

Criterio: deve fallire se rileva future leak, incoerenze, provenance mancante.

18.6 5) Forecast ranking (Wave 6)

```
py scripts/sentinel.py forecast
# oppure tool diretto
```

```
py -m src.tools.forecast_rankings --help
```

Output atteso: JSON serializzabile e/o report in reports/ (a seconda del tool).

Parametri chiave: vedi 007_PARAMETER_SNAPSHOT.md.

18.7 6) NEWS-ALPHA (lane news)

18.7.1 6.1 Status

```
py scripts/news_alpha.py status
```

18.7.2 6.2 Collect (RSS) - online guardato

Online richiede:

- --online
- NEWS_ALPHA_ALLOW_ONLINE=1 oppure --allow-online

Esempio:

```
set NEWS_ALPHA_ALLOW_ONLINE=1
```

```
py scripts/news_alpha.py collect --online --allow-online --date-from 2026-01-01 --date-to 2026-01-07
```

Input aggiuntivi della collect:

- query window when-days
- allowlist domini (opzionale)

Output attesi della collect:

- raw RSS XML (se online) in reports/news_alpha/raw/rss
- fixtures JSONL in reports/news_alpha/collector/collector_<ts>.jsonl
- stats JSON con conteggi e qualità

Gate opzionale:

- --strict-dq (fallisce se zero items “kept”)

18.7.3 6.3 Run (consume fixtures -> DuckDB)

```
py scripts/news_alpha.py run --date-from 2026-01-01 --date-to 2026-01-07 --fixtures <path_to_jsonl>
```

Rating discreto derivato da score medio:

- BUY se score ≥ 0.20
- DOWNGRADE se score ≤ -0.20
- HOLD altrimenti

Artefatti attesi:

- fixtures JSONL in reports/news_alpha/...
 - eventuali rejects JSONL (dati scartati con motivazione)
 - righe in recs e sentiment_cache
-

18.7.4 6.4 History lane (GDELT)

Il runner include comandi history per alimentare dataset storici (sempre con guardrail online):

```
py scripts/news_alpha.py history download
```

```
py scripts/news_alpha.py history profile
```

```
py scripts/news_alpha.py history fixtures
```

Nota: richiede --online + allow (policy/env) come per RSS.

18.8 7) Avvio UI Streamlit

```
py -m streamlit run app.py
```


Le pagine UI sono in `pages/`.

18.9 8) Packaging sessione (archiviazione deterministica)

```
py scripts/pack_session.py --help
py scripts/ops_run_session.py --help
```

Obiettivo: consolidare una sessione (`run/certify/report`) in cartelle deterministiche, evitando symlink.

18.10 9) Naming convention (artefatti)

- Transcript:
 - `reports/RUN_TRANSCRIPT_<run_id>.txt`
 - `reports/CERTIFY_TRANSCRIPT_<run_id>.txt`
 - NEWS-ALPHA:
 - `reports/news_alpha/collector/collector_<ts>.jsonl`
 - `reports/news_alpha/rejects_<ts>.jsonl`
 - DB:
 - `data/sentinel_alpha.db` (single source of truth locale)
-

18.11 10) Checklist di riproducibilit  (retail)

- Stessa versione codice (verificare `code_fingerprint`)
- Stesso DB snapshot o DB rigenerato con stesso input
- Offline (per risultati “certification-grade”) salvo necessit  di backfill esplicito
- Report sempre allegati alla sessione (pack)

\newpage

19 009_GAP_REGISTER.md

20 Gap Register - v1.1

Build date: 2026-01-25

20.1 Scopo

Elencare in modo esplicito:

- cosa   **implementato** nello snapshot
- cosa   **parzialmente implementato** (con disclosure)
- cosa   **mancante** e deve entrare in backlog

Questo documento   parte della tracciabilit  e serve a evitare che il PDF “prometta” funzionalit  non presenti.

20.2 1) Implementato (con evidenza nel codice)

- Runner SENTINEL-ALPHA: `migrate/test/run/certify/verify/forecast/status` (`scripts/sentinel.py`)
- Runner NEWS-ALPHA: `collect/run/status + history lane` (`scripts/news_alpha.py`)
- DuckDB schema owner + migrazioni (`src/db/migrate.py`, schema v2.1.0)
- Audit engine: trade ledger + equity curve + reporting (`src/core/audit_engine.py`, `src/report_generator.py`)
- Controlli: `verify_*` tools (provenance, ticker mappings, inputs)
- Forecast ranking a stelle (Wave 6): shrinkage, confidence, percentile-to-stars (`src/forecast/ranking.py`)
- UI Streamlit multi-pagina (`app.py`, `pages/*.py`)
- Modelli retail: cost model e tax model (Italia) con loss carry (semplificato)

20.3 2) Parzialmente implementato / disclosure-first

- **Dividendi**: tabella dividends presente; cashflow total return e' indicato come estensione incrementale.
- **Disclosure switches** (SENTINEL_DIVIDEND_POLICY, SENTINEL_TIMING_MODE): impostati dal runner; non tutte le ramificazioni economiche sono necessariamente attive in ogni modulo.
- **Provider price backfill**: supportato e loggato (data_gaps), ma dipende da configurazione e rete (online guardato).

20.4 3) Mancante (backlog suggerito)

20.4.1 Execution layer (paper/live) + OMS + log ordini

- Entry point (es. scripts/execute.py) che consuma FORECAST_RANKING_LATEST.json (o equivalente) e produce ordini.
- Gateway verso broker API (paper-first; live solo con flag esplicito e secret via env).
- Persistenza di lifecycle ordine/fill in DB (execution_log) per audit trail completo.
- Position keeping (stato posizioni) per evitare mismatch tra backtest e operativita' reale.

20.4.2 Risk management (pre-trade e post-trade)

- RiskEngine con:
 - limiti di concentrazione (per ticker/settore/asset class)
 - position sizing (volatility targeting / risk_scalar)
 - stop-loss (es. ATR-based) e kill-switch su drawdown
 - stress test e liquidity cap
- Output obbligatorio: allow/deny per ordine con reason_code e parametri applicati.

20.4.3 Transaction Cost Analysis (TCA) + model monitoring (drift)

- Misura differenza tra “simulato” (audit_engine) e “realizzato” (execution_log): slippage, fee drag, fill quality.
- Monitor forecast vs realized su finestre rolling (IC, hit-rate, drawdown, turnover, cost drag) e alert su soglie.
- Surface in UI (risk flags) e in report (disclosure).

20.4.4 Upgrade News/Sentiment (opzionale, se richiesto)

- Modelli NLP piu' espressivi (es. transformer fine-tuned) mantenendo vincoli di riproducibilita' (snapshot/fingerprint, caching).
- Integrazione di ulteriori fonti (earnings transcripts, social media filtrati) solo se gestibili con provenance e licensing.

20.4.5 Macro/monetary layer (trading avanzato, taglio retail)

- Dataset tassi (policy rate), inflazione e/o curve (anche via fonti pubbliche).
- Regime detection (risk-on/off) per modulare rischio e confidence.

20.4.6 FX translation e multi-currency

- Conversione PnL in valuta base (EUR) per tickers US vs EU.
- Tabella FX rates + integration in equity curve.

20.4.7 Corporate actions end-to-end

- Split/merge adjustments e dividendi integrati nella performance (total return).

20.4.8 Execution realism avanzato

- Slippage/spread dinamico per liquidita' e market cap.
- Limiti size retail e fractional shares.

20.5 4) Documentazione: gap residui

- Nessuno “gap strutturale” bloccante: PDR/PDD/DataDictionary sono presenti come base.
- Prossimo passo: consolidare e versionare formalmente (tag docs/ come “canonico”) e decidere naming standard (es. 000_...).

\newpage